นี่คือโครงสร้าง FastAPI แบบ Local First ที่พร้อม deploy ได้ทั้ง Vercel (Serverless) และ Kubernetes (Production) โดยไม่ต้องแก้โครงสร้างแอปใหม่

📁 โครงสร้างโปรเจค

```
fastapi-oauth-k8s/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── core/
│   │   ├── __init__.py
│   │   └── config.py
│   ├── api/
│   │   ├── __init__.py
│   │   ├── auth.py
│   │   ├── callback.py
│   │   └── health.py
```

```
│   └── services/
│       ├── __init__.py
│       └── token_service.py
├── k8s/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── ingress.yaml
├── .github/
│   └── workflows/
│       └── ci-cd.yml
├── Dockerfile
├── docker-compose.yml
├── vercel.json
├── requirements.txt
└── .env.example
```

---

01 app/main.py

python

```python
from fastapi import FastAPI
from fastapi.middleware.cors import
CORSMiddleware
from app.api import auth, callback, health
from app.core.config import settings

app = FastAPI(
    title="OAuth2 PKCE API",
    version="1.0.0",
    docs_url="/docs" if not settings.ENV ==
"production" else None,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth.router, prefix="/
```

```
auth", tags=["Auth"])
app.include_router(callback.router,
prefix="/auth", tags=["Auth"])
app.include_router(health.router, prefix="/
health", tags=["Health"])

@app.get("/")
async def root():
    return {
        "message": "FastAPI OAuth2 PKCE",
        "environment": settings.ENV,
        "callback_url":
settings.OAUTH_CALLBACK_URL,
    }
```

---

02 app/core/config.py

python

```
from pydantic_settings import
```

```
BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    ENV: str = "local"  # local | vercel |
production

    # OAuth2 Provider (Google, GitHub,
etc.)
    OAUTH_CLIENT_ID: str
    OAUTH_CLIENT_SECRET: str | None =
None  # PKCE ไม่ต้องใช้ secret บน client
    OAUTH_AUTHORIZE_URL: str = "https://
accounts.google.com/o/oauth2/v2/auth"
    OAUTH_TOKEN_URL: str = "https://
oauth2.googleapis.com/token"
    OAUTH_USERINFO_URL: str = "https://
openidconnect.googleapis.com/v1/
userinfo"

    # Callback URL แยกตาม Environment
    @property
    def OAUTH_CALLBACK_URL(self) -> str:
        callbacks = {
            "local": "http://localhost:8000/auth/
```

```
callback",
            "vercel": "https://your-
app.vercel.app/auth/callback",
            "production": "https://
api.example.com/auth/callback",
        }
        return callbacks.get(self.ENV,
callbacks["local"])

    # Frontend redirect หลัง login สำเร็จ
    @property
    def FRONTEND_URL(self) -> str:
        urls = {
            "local": "http://localhost:3000",
            "vercel": "https://your-
app.vercel.app",
            "production": "https://
app.example.com",
        }
        return urls.get(self.ENV, urls["local"])

    # CORS
    @property
    def CORS_ORIGINS(self) -> list[str]:
        return [self.FRONTEND_URL]
```

```
    # JWT
    JWT_SECRET: str = "super-secret-
change-in-production"
    JWT_ALGORITHM: str = "HS256"
    JWT_EXPIRE_MINUTES: int = 60

    # Redis (optional - for token storage)
    REDIS_URL: str | None = None

    class Config:
        env_file = ".env"
        case_sensitive = True

@lru_cache()
def get_settings() -> Settings:
    return Settings()

settings = get_settings()
```

---

03 app/api/auth.py

python

```
import secrets
import hashlib
import base64
from fastapi import APIRouter, Request
from fastapi.responses import
RedirectResponse
from app.core.config import settings

router = APIRouter()

def generate_pkce() -> tuple[str, str]:
    """Generate PKCE code_verifier and
code_challenge"""
    verifier = base64.urlsafe_b64encode(
        secrets.token_bytes(32)
    ).decode("utf-8").rstrip("=")
    challenge = base64.urlsafe_b64encode(

hashlib.sha256(verifier.encode()).digest()
```

```
    ).decode("utf-8").rstrip("=")
    return verifier, challenge

@router.get("/login")
async def login(request: Request):
    """Initiate OAuth2 + PKCE flow"""
    verifier, challenge = generate_pkce()

    # Store verifier in session/cookie
(simplified - use Redis in production)
    state = secrets.token_urlsafe(16)

    auth_url = (

f"{settings.OAUTH_AUTHORIZE_URL}"
        f"?
client_id={settings.OAUTH_CLIENT_ID}"

f"&redirect_uri={settings.OAUTH_CALLBA
CK_URL}"
        f"&response_type=code"
        f"&scope=openid email profile"
        f"&state={state}"
        f"&code_challenge={challenge}"
        f"&code_challenge_method=S256"
```

```
    )

    response =
RedirectResponse(url=auth_url)
    response.set_cookie(
        key="oauth_state",
        value=state,
        httponly=True,
        secure=settings.ENV != "local",
        samesite="lax",
    )
    response.set_cookie(
        key="code_verifier",
        value=verifier,
        httponly=True,
        secure=settings.ENV != "local",
        samesite="lax",
    )
    return response
```

---

04 app/api/callback.py

python

```
import httpx
from fastapi import APIRouter, Request,
HTTPException
from fastapi.responses import
RedirectResponse
from app.core.config import settings
from app.services.token_service import
create_jwt_token

router = APIRouter()

@router.get("/callback")
async def auth_callback(request:
Request, code: str, state: str | None =
None):
    """Handle OAuth2 callback and
exchange code for token"""
    # Verify state
    stored_state =
```

```
request.cookies.get("oauth_state")
    if stored_state and state !=
stored_state:
        raise
HTTPException(status_code=400,
detail="Invalid state parameter")

    verifier =
request.cookies.get("code_verifier")
    if not verifier:
        raise
HTTPException(status_code=400,
detail="Missing code verifier")

    # Exchange code for access token
    token_data = {
        "client_id":
settings.OAUTH_CLIENT_ID,
        "client_secret":
settings.OAUTH_CLIENT_SECRET,
        "code": code,
        "redirect_uri":
settings.OAUTH_CALLBACK_URL,
        "grant_type": "authorization_code",
        "code_verifier": verifier,
```

```
    }

    async with httpx.AsyncClient() as client:
        token_resp = await
client.post(settings.OAUTH_TOKEN_URL,
data=token_data)
        if token_resp.status_code != 200:
            raise
HTTPException(status_code=400,
detail="Token exchange failed")

        tokens = token_resp.json()
        access_token =
tokens.get("access_token")

        # Get user info
        user_resp = await client.get(
            settings.OAUTH_USERINFO_URL,
            headers={"Authorization": f"Bearer
{access_token}"}
        )
        user_info = user_resp.json()

    # Create JWT for your app
    jwt_token = create_jwt_token({
```

```
        "sub": user_info.get("email"),
        "name": user_info.get("name"),
        "picture": user_info.get("picture"),
    })

    # Redirect to frontend with token
    redirect_url =
f"{settings.FRONTEND_URL}/auth/
success?token={jwt_token}"
    response =
RedirectResponse(url=redirect_url)
    response.delete_cookie("oauth_state")
    response.delete_cookie("code_verifier")
    return response
```

---

05 app/api/health.py

python

```
from fastapi import APIRouter
```

```
from pydantic import BaseModel

router = APIRouter()

class HealthResponse(BaseModel):
    status: str
    environment: str

@router.get("",
response_model=HealthResponse)
async def health_check():
    from app.core.config import settings
    return HealthResponse(
        status="healthy",
        environment=settings.ENV,
    )

@router.get("/ready")
async def readiness_check():
    return {"ready": True}

@router.get("/live")
async def liveness_check():
    return {"alive": True}
```

---

06 app/services/token_service.py

python

```
from datetime import datetime,
timedelta, timezone
import jwt
from app.core.config import settings

def create_jwt_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) +
timedelta(minutes=settings.JWT_EXPIRE
_MINUTES)
    to_encode.update({"exp": expire, "iat":
datetime.now(timezone.utc)})
    return jwt.encode(to_encode,
settings.JWT_SECRET,
algorithm=settings.JWT_ALGORITHM)
```

```
def decode_jwt_token(token: str) -> dict:
    return jwt.decode(token,
settings.JWT_SECRET,
algorithms=[settings.JWT_ALGORITHM])
```

---

07 Dockerfile

```dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r
requirements.txt

# Copy app
COPY app/ ./app/
```

```
# Non-root user for security
RUN useradd -m appuser && chown -R
appuser:appuser /app
USER appuser

# Expose port
EXPOSE 8000

# Run with uvicorn (single process for
Vercel compatibility)
CMD ["uvicorn", "app.main:app", "--host",
"0.0.0.0", "--port", "8000"]
```

---

08 docker-compose.yml

```yaml
version: "3.8"
```

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    environment:
      - ENV=local
    volumes:
      - ./app:/app/app  # Hot reload for dev
    command: uvicorn app.main:app --host
0.0.0.0 --port 8000 --reload

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

---

09 vercel.json

json


```
{
  "builds": [
    {
      "src": "app/main.py",
      "use": "@vercel/python"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "app/main.py"
    }
  ],
  "env": {
    "ENV": "vercel"
  }
}
```

⚠️ หมายเหตุ: Vercel รองรับ Python Serverless Functions แต่ไม่รองรับ WebSocket และ background tasks แบบ long-running

---

10 k8s/deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fastapi-oauth
  labels:
    app: fastapi-oauth
spec:
  replicas: 3
  selector:
    matchLabels:
      app: fastapi-oauth
  template:
```

```
    metadata:
      labels:
        app: fastapi-oauth
    spec:
      containers:
        - name: api
          image: ghcr.io/YOUR_USERNAME/
fastapi-oauth:latest
          ports:
            - containerPort: 8000
          env:
            - name: ENV
              value: "production"
          envFrom:
            - secretRef:
                name: fastapi-secrets
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "512Mi"
              cpu: "500m"
          livenessProbe:
            httpGet:
```

```
        path: /health/live
        port: 8000
      initialDelaySeconds: 10
      periodSeconds: 10
    readinessProbe:
      httpGet:
        path: /health/ready
        port: 8000
      initialDelaySeconds: 5
      periodSeconds: 5
```

---

11 k8s/service.yaml

yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: fastapi-oauth-service
spec:
```

```
  selector:
    app: fastapi-oauth
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
  type: ClusterIP
```

---

12 k8s/ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: fastapi-oauth-ingress
  annotations:
    cert-manager.io/cluster-issuer:
"letsencrypt-prod"
    nginx.ingress.kubernetes.io/ssl-
```

```
  redirect: "true"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - api.example.com
      secretName: api-tls-secret
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: fastapi-oauth-service
                port:
                  number: 80
```

13-14 GitHub Actions CI + Docker → GHCR

yaml

```yaml
# .github/workflows/ci-cd.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: $`{{ github.repository }}

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
```

```
      with:
        python-version: "3.11"

    - name: Install dependencies
      run: |
        pip install -r requirements.txt
        pip install pytest httpx

    - name: Run tests
      run: pytest || echo "No tests yet"

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to GHCR
```

```
      uses: docker/login-action@v3
      with:
        registry: ${{ env.REGISTRY }}
        username: ${{ github.actor }}
        password: ${{
secrets.GITHUB_TOKEN }}

    - name: Extract metadata
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${{ env.REGISTRY }}/${{
env.IMAGE_NAME }}

    - name: Build and push
      uses: docker/build-push-action@v5
      with:
        context: .
        push: true
        tags: |
          ${{ env.REGISTRY }}/${{
env.IMAGE_NAME }}:latest
          ${{ env.REGISTRY }}/${{
env.IMAGE_NAME }}:${{ github.sha }}
        cache-from: type=gha
```

```
          cache-to: type=gha,mode=max

  deploy-vercel:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to Vercel
        uses: vercel/action-deploy@v1
        with:
          vercel-token: `${{
secrets.VERCEL_TOKEN }}
          vercel-org-id: $`{{
secrets.VERCEL_ORG_ID }}
          vercel-project-id: `${{
secrets.VERCEL_PROJECT_ID }}
```

---

15 Vercel Deployment

```bash
# ติดตั้ง Vercel CLI
npm i -g vercel

# Login
vercel login

# Link project
vercel link

# Deploy
vercel --prod
```

Environment Variables บน Vercel Dashboard:

- ENV=vercel
- OAUTH_CLIENT_ID=xxx
- OAUTH_CLIENT_SECRET=xxx
- JWT_SECRET=xxx

---

16 Kubernetes Deployment

```bash
# 1. สร้าง Secrets
kubectl create secret generic fastapi-secrets \
  --from-literal=OAUTH_CLIENT_ID=xxx \
  --from-literal=OAUTH_CLIENT_SECRET=xxx \
  --from-literal=JWT_SECRET=xxx

# 2. Deploy
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/ingress.yaml

# 3. Verify
kubectl get pods
kubectl logs -l app=fastapi-oauth
```

---

17 Prometheus/Grafana (Optional)

yaml

```yaml
# k8s/monitoring.yaml
apiVersion: v1
kind: ServiceMonitor
metadata:
  name: fastapi-metrics
spec:
  selector:
    matchLabels:
      app: fastapi-oauth
  endpoints:
    - port: metrics
      path: /metrics
      interval: 15s
```

เพิ่ม prometheus-fastapi-instrumentator ใน requirements.txt และติดตั้งใน main.py:

```python
from prometheus_fastapi_instrumentator import Instrumentator

Instrumentator().instrument(app).expose(app, endpoint="/metrics")
```

---

18 Secrets Management

Environment	Method
Local	.env file
Vercel	Dashboard → Settings → Environment

Variables

K8s kubectl create secret หรือ External Secrets Operator

CI/CD GitHub Secrets (Settings → Secrets and variables)

---

requirements.txt

```txt
fastapi==0.109.0
uvicorn[standard]==0.27.0
pydantic-settings==2.1.0
httpx==0.26.0
pyjwt==2.8.0
python-jose[cryptography]==3.3.0
prometheus-fastapi-instrumentator==6.1.0
```

__init__.py ที่จำเป็น

สร้างไฟล์ว่างใน:

- app/__init__.py
- app/core/__init__.py
- app/api/__init__.py
- app/services/__init__.py

🎯 สรุปจุดสำคัญ

Feature Implementation
PKCE code_verifier + code_challenge แบบ SHA256
Callback แยก Env

settings.OAUTH_CALLBACK_URL ผ่าน @property

Local Dev docker-compose up มี hot reload

Vercel vercel.json + Serverless Functions

K8s Health probes + GHCR image + Ingress TLS

Security HttpOnly cookies + State validation + JWT

พร้อมใช้งานแล้ว! แค่เปลี่ยน YOUR_USERNAME, api.example.com, และ OAuth Provider settings ตามต้องการ 🚀


23:19
ไม่สามารถแชร์การสนทนาได้
ไม่สามารถแชร์รูปภาพและไฟล์ที่อัปโหลดได้
YAML Syntax
Skip to content ZyntroAI fastapi-python-boilerplate Repository navigation Code Issues 3 (3) Pull requests 4 (4) Agents Discussions Actions Projects Security and quality 43 (43) Insights Settings .github/ workflows/Auto-Index-Sync.yml Update README.md #699 All jobs Run details Workflow file for this run .github/workflows/ Auto-Index-Sync.yml at 2cba7c3 name: Auto Index Sync # ⏰ Triggers — Push, Nightly
แชร์
ทุกคนที่มีลิงก์นี้สามารถดูและแชร์การสนทนานี้ได้ ตรวจสอบข้อมูลที่ละเอียดอ่อนมีก่อนแชร์


23:19
ไม่สามารถแชร์การสนทนาได้
ไม่สามารถแชร์รูปภาพและไฟล์ที่อัปโหลดได้
YAML Syntax
Skip to content ZyntroAI fastapi-python-boilerplate Repository navigation Code Issues 3 (3) Pull requests 4 (4) Agents Discussions Actions Projects Security and quality 43 (43) Insights Settings .github/ workflows/Auto-Index-Sync.yml Update README.md #699 All jobs Run details Workflow file for this run .github/workflows/ Auto-Index-Sync.yml at 2cba7c3 name: Auto Index Sync # ⏰ Triggers — Push, Nightly
แชร์
ทุกคนที่มีลิงก์นี้สามารถดูและแชร์การสนทนานี้ได้ ตรวจสอบข้อมูลที่ละเอียดอ่อนมีก่อนแชร์

Ccccc